

Module 1

INTRODUCTION TO GIT & GITHUB

Visual & Activity-Based Teaching Presentation

Dr. R. Varalakshmi

Dean of Academics, Associate Professor and Head
M.Sc. Data Science and Artificial Intelligence

START → AGENDA

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **Explain why version control is needed.**
- Identify Git as a version-control system and GitHub as an online collaboration platform.
- Differentiate Git and GitHub using a simple real-life example.
- Recognize repository, commit, branch, merge, history, pull request and README.
- Create a GitHub account and explore the dashboard.
- Choose Public or Private visibility based on project purpose.

Unit 1 Topics

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

01 Why Version Control

Problem → solution → benefits

02 Introduction to Git

Concepts and workflow

03 What is GitHub?

Platform and collaboration

04 Git vs GitHub

Tool vs platform

05 GitHub Features

Core features

06 GitHub Interface

Repository navigation

07 Public vs Private

Visibility choice

08 GitHub Use Cases

Education → research → business

09 Activity 1

Create account

10 Activity 2

Explore dashboard

CLICK A TOPIC TO JUMP DIRECTLY TO IT

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



Why Version Control?

Keep your work safe. Track changes. Work smarter together.



WHAT IS VERSION CONTROL?

Version control is a system that records every change made to files over time so you can go back to any previous version whenever needed.



THE PROBLEM WITH TRADITIONAL FILE STORAGE

When we save files without version control...



W	Project.docx	?
W	Project_Final.docx	?
W	Project_Final2.docx	?
W	Project_ReallyFinal.docx	?
W	Project_New_Final.docx	?
W	Project_UseThisOne.docx	?

- ⚠ Hard to know the latest version
- 🕒 Easy to lose or overwrite files
- 🗑 No history of changes

- 👥 Difficult to work in a team
- 🔍 Time wasted searching files

VERSION CONTROL IS THE SOLUTION!

Version control systems help you:

- ✅ Save every change automatically
- ✅ Go back to any previous version
- ✅ Track who changed what and when
- ✅ Collaborate with others easily
- ✅ Keep your work safe and organized



**One project.
All versions.
Always in order.**

BENEFITS OF VERSION CONTROL



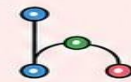
SAFE & SECURE
Your work is backed up and protected from loss.



TRACK CHANGES
See every change made over time with full history.



EASY COLLABORATION
Many people can work together without conflicts.



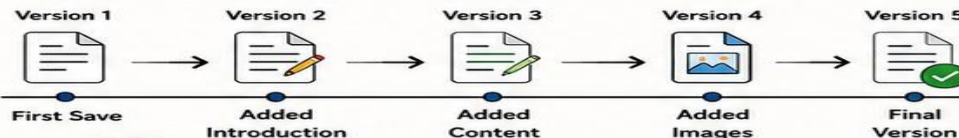
EXPERIMENT FREELY
Try new ideas in branches without affecting main work.



PRODUCTIVITY BOOST
Less time fixing mistakes, more time building great things!

REAL-LIFE ANALOGY

Think of version control like a **Time Machine** for your project.



You can go back to any version whenever you want!



KEY TAKEAWAY

Version control helps you organize, track, and protect your work.
It's a smart habit for every student and every project!



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

Imagine Project.docx, Project_Final.docx, Project_Final2.docx and Project_ReallyFinal.docx.

The latest file is difficult to identify.

Files can be overwritten, duplicated or lost.

There is no reliable history of who changed what and when.

Teamwork becomes confusing and time is wasted.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

Version control records changes to files over time.

You can return to an earlier version instead of starting again.

One project can have an organized history of many versions.

Changes can be reviewed and tracked.

Think of it as a TIME MACHINE for your project.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

1. Edit your file.

2. Save a meaningful version of the changes.

3. The system stores a snapshot in the project history.

4. View history whenever needed.

5. Restore an earlier version if something goes wrong.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



INTRODUCTION TO GIT

Git is a Version Control System that helps you track changes in your files over time.



WHAT IS GIT?

Git is a free and open-source **distributed** version control system.

- ✓ Tracks every change you make.
- ✓ Lets you go back to any previous version.
- ✓ Helps many people work on the same project easily.
- ✓ Works offline on your computer.



HOW DOES GIT WORK?



You edit your file.



You add the changes.



You commit the changes.



Git saves a snapshot (history).

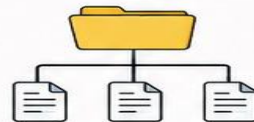


Git remembers every snapshot, so you can compare and restore anytime.

KEY CONCEPTS IN GIT

1. REPOSITORY

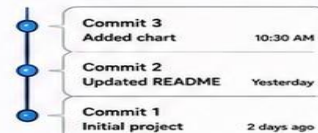
A folder where your project files and their history are stored.



Example: My-Project

2. COMMIT

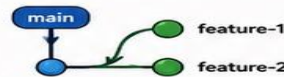
A saved snapshot of your changes with a message.



Example: "Added student info"

3. BRANCH

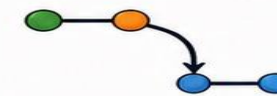
A separate line of work. You can try new things without affecting the main project.



Example: feature-login

4. MERGE

Combining changes from one branch into another.



Example: Merge feature-1 into main

5. HISTORY

The list of all commits (snapshots) in order.



Example: View commit history

WHERE DOES GIT WORK?

Git can work in two places:

LOCAL (On your computer)



You can create repositories, make changes and commit without internet.

REMOTE (Online – e.g., GitHub)



You can upload (push) your work online and share with others.

BASIC GIT WORKFLOW

- 1 Create / Get a repository
Start a new project.
- 2 Make changes
Edit your files.
- 3 Add changes
Stage the changes.
- 4 Commit changes
Save with a message.
- 5 Review history
See all your changes.

WHY GIT IS IMPORTANT?

- Protects your work**
No more fear of losing files.
- Tracks every change**
You know what, when and who changed.
- Supports teamwork**
Many people can work together smoothly.
- Encourages experiments**
Create branches and try new ideas safely.
- Professional skill**
Git is widely used in schools, companies and research.



KEY TAKEAWAY

Git = Track changes + Save versions + Collaborate easily
It is the time machine for your projects!



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **Repository** — project folder containing files and their history.
- **Commit** — saved snapshot of changes with a message.
- **Branch** — separate line of work for a new feature or experiment.
- **Merge** — combine changes from one branch into another.
- **History** — ordered list of commits.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **Create or get a repository.**
- Make changes to project files.
- Add or stage the changes.
- Commit the changes with a clear message.
- Review history and continue working.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **LOCAL** — Git can manage a repository on your computer.
- **REMOTE** — a Git repository can also be stored online, such as on GitHub.
- Many Git operations can be performed locally.
- Later, changes can be synchronized with a remote repository.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



WHAT IS GITHUB?



GitHub is an online platform where people can **store, manage, share, and work together** on software projects.



THINK OF GITHUB LIKE...

GitHub is like **Google Drive** for coding projects, but with special tools for programmers.



Store your code



Keep history of changes



Work together with others



Build amazing projects

KEY FEATURES OF GITHUB



Cloud-Based

Access your projects from anywhere, anytime.



Collaboration

Work with others on the same project in real-time.



Version Control

Track every change. Go back to any previous version.



Safe & Secure

Your code is safe in the cloud with backup.



Open Source

Share your projects with the world and learn from others.

IMPORTANT TERMS TO KNOW



Repository

A project folder where your code and files are stored.



Code

The files and programs you write.



Version History

A record of all the changes made to your project.



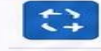
Collaboration

Working with others by sharing and making changes together.



Branch

A separate line of development to work on new features.



Pull Request

A request to review and merge your changes into the main project.



README

A file that explains what your project is about.

WHY USE GITHUB?

- ✓ Helps in organizing your projects.
- ✓ Makes teamwork easy.
- ✓ Tracks every change safely.
- ✓ Builds your portfolio as a developer.
- ✓ Used by millions of developers worldwide!



EXAMPLE

If 5 students are building an AI project:

- They put the project on GitHub.
- Each student works on different parts.
- They track changes and help each other.
- Finally, they combine everything into one complete project!



ONE-LINE TO REMEMBER

GitHub = A cloud-based platform for **storing, managing, sharing, and collaborating** on coding projects.



REMEMBER:

Write Code. Save on GitHub. Share. Collaborate. Succeed!

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

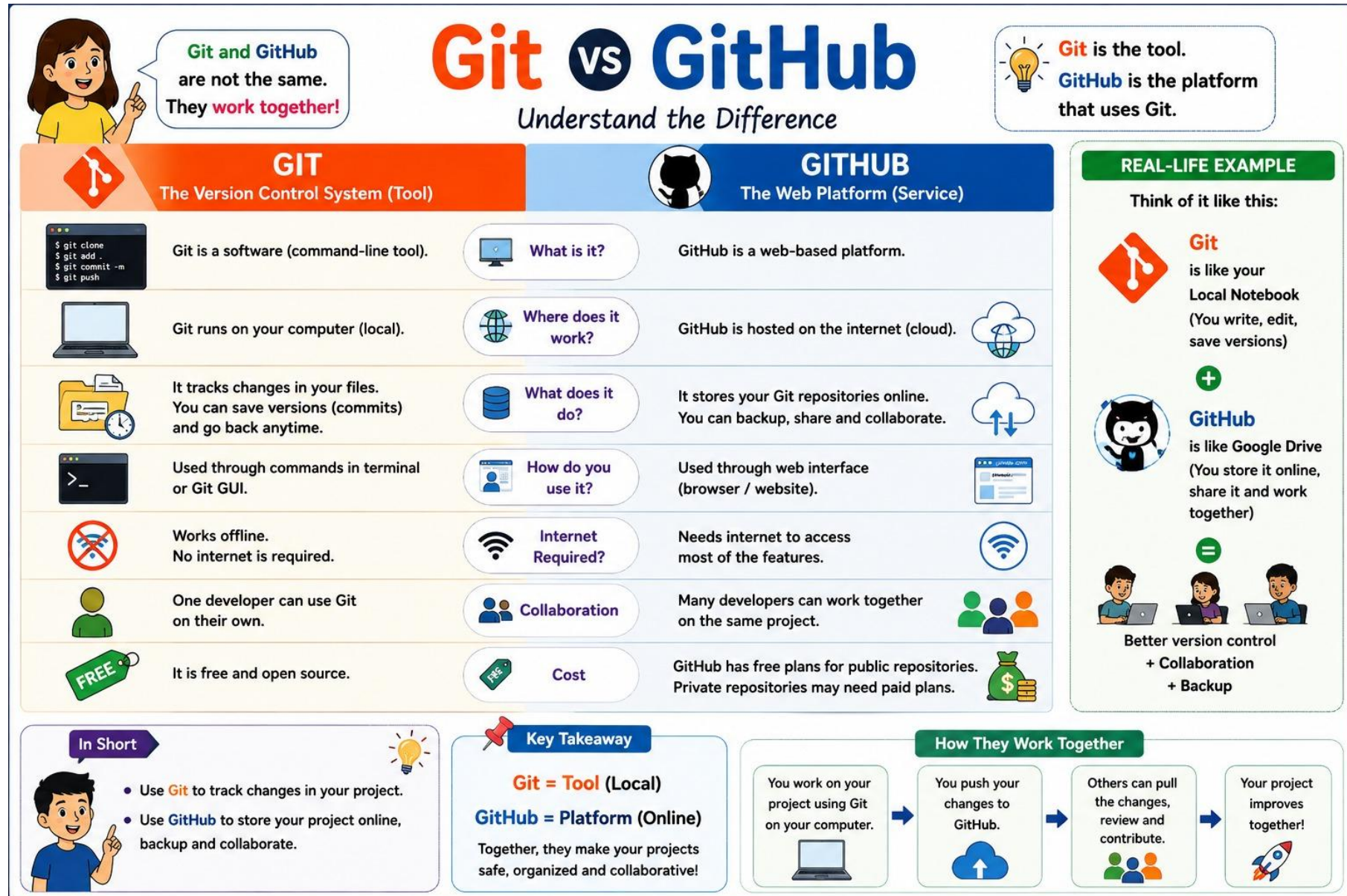
7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **Git = the version-control tool that tracks changes.**
- GitHub = the online platform where Git repositories can be stored and shared.
- You work on the project using Git on your computer.
- You push changes to GitHub.
- Other people can pull, review and contribute.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



GitHub Features

Powerful tools to host, manage and collaborate on your code projects.



1 Repositories



Store your project files, track changes and manage versions.

my-project

- README.md
- index.html
- style.css
- script.js

2 Commits & History



Save changes (commits) with messages and view full history.

Commits

- Update README 2 hours ago
- Add index.html yesterday
- Initial commit 3 days ago

3 Branches



Create branches to work on new features without affecting main code.

- main
- feature-login
- bug-fix

4 Pull Requests



Review code, discuss changes and merge improvements.

Open Pull Request

feature-login → main

✓ Checks passed

2 reviewers

5 Issues



Track bugs, tasks and ideas. Organize work effectively.

- #23 Login bug bug
- #24 Add dark mode enhancement
- #25 Improve UI task

6 Discussions



Have conversations, ask questions and share ideas.

How to contribute?

Answer

Ideas for new features?

5 replies

5 replies

7 Projects



Plan your work using boards, lists and cards.

Project Board

To Do	In Progress	Done
bug	enhancement	documentation
good first issue		

8 Labels



Use labels to categorize and filter issues and pull requests.

- bug
- enhancement
- documentation
- good first issue

9 Milestones



Set milestones to track progress towards important goals.

v1.0 Release	60%
Website Launch	20%

10 Actions (CI/CD)



Automate testing, building and deployment with GitHub Actions.

Build and Test

- ✓ Set up
- ✓ Run tests
- Deploy

11 GitHub Pages



Publish websites directly from your repository for free.

https://username.github.io

12 Security



Keep your code safe with security alerts and vulnerability tracking.

- Dependabot alerts
- Code scanning
- Secret scanning

Why These Features Matter?



Better Collaboration
Work together seamlessly.



Save Time
Automate and organize your work.



Safe & Secure
Protect your code and data.



Build & Deploy
Ship your projects faster.



Track Progress
Stay on track with clear goals.

Used For



Education
Student projects, assignments, collaboration.



Research
Share code, data and papers openly.



Business
Team development, product building and DevOps.



Personal Portfolio
Showcase your skills and projects.



Open Source
Contribute to projects that help the world.



GitHub brings everything together – **Code, Collaboration and Community** – in one platform.
Learn it. Use it. Build something amazing!



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **Repositories — store project files and history.**
- **Commits & History — record changes.**
- **Branches — develop new ideas safely.**
- **Pull Requests — review and merge changes.**
- **Issues & Discussions — track tasks, bugs and questions.**
- **Projects, Actions, Pages and Security — support planning, automation, publishing and protection.**

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



GitHub is where developers store, manage and collaborate on code.

GITHUB INTERFACE

Get to know the main parts of GitHub!

What is GitHub?

GitHub is a web-based platform that uses Git to store code, track changes and work together on projects.



1 Top Navigation Bar

- Search repositories
- Go to Pull requests, Issues, Marketplace, Explore
- Access notifications and your profile

2 Repository Path

Shows the owner / user name and repository name.

3 Watch / Star / Fork

- Watch** : Get updates
- Star** : Show support
- Fork** : Copy the repo to your account

4 Tabs

Different sections of the repository.

- Code** – files and folders
- Issues** – bugs, tasks
- Pull requests** – review changes
- Actions** – workflows
- Projects** – boards
- Wiki** – documentation
- Security** – security info
- Insights** – statistics

5 File & Folder Area

Shows all files and folders in the repository. Click any file to view its content.

6 About Section

Shows project information, links, license, stars, watchers and forks.

7 README Preview

README.md is the main file that appears on the front page of your repository.

8 Code Button

Clone, download or open the repository using HTTPS, SSH or GitHub CLI.

Other Important Areas

Notifications See alerts for mentions, issues, pull requests and more.	Create Create new repository, new file, organization, or project.	Profile Menu Access settings, your repositories, gists, switch accounts.	Branch Selector View and switch branches. main is the default branch.	Release / Tags View all released versions and tags of the project.	Insights See traffic, clones, popular content and more stats.
---	--	---	--	---	--

TIP

- ★ Star repositories you like.
- 👁 Watch to get updates.
- 🔗 Fork to experiment freely.
- 💬 Use Issues and Discussions to communicate.

KEY TAKEAWAY



GitHub's interface helps you manage your project, collaborate with others and build amazing things together!



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **Repository path tells you the owner and repository name.**
- Tabs provide access to Code, Issues, Pull Requests, Actions, Projects, Wiki, Security and Insights.
- File and folder area shows the project contents.
- README.md explains the project on the repository front page.
- The Code button provides ways to access or clone the repository.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



Public vs Private Repositories

Two types of GitHub repositories to store and share your code.

Both are Git repositories but differ in visibility and access.

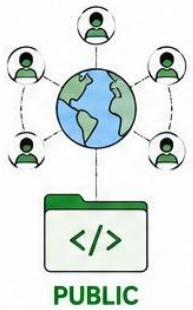


PUBLIC REPOSITORY

Anyone on the internet can **see** your repository.

Key Points

- Visible to **everyone**. Anyone can view, clone and fork your repository.
- Great for **Open Source** projects. Encourages collaboration and community contribution.
- Anyone can **fork** your repo and suggest improvements (via Pull Requests).
- Builds your profile and **portfolio**. Shows your work to potential employers and collaborators.
- FREE** Free to use. No cost for creating or using public repositories.



VS

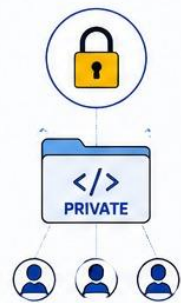


PRIVATE REPOSITORY

Only you and **people you choose** can see your repository.

Key Points

- Visible only to **you and collaborators**. Keeps your code and ideas confidential.
- Ideal for **private projects** like company work, client projects, or personal ideas.
- You control **access**. Invite specific people to view or contribute.
- More **secure** for sensitive information like business logic, APIs, or proprietary code.
- Requires a **paid plan** on GitHub (Free accounts can't create private repositories).



USE CASES – PUBLIC

- Open Source Software**
Share your code and help others.
- Student Projects**
Showcase your skills and learning.
- Tutorials & Examples**
Help others by sharing examples.

QUICK COMPARISON

Feature	Public Repository	Private Repository
Visibility	Everyone on the internet	Only you and invited people
Access	Anyone can view, clone and fork	Only invited people can view and contribute
Best For	Open Source projects, Learning, Portfolio	Company projects, Client work, Personal ideas
Cost	Free	Paid plan required
Forking	Anyone can fork	Only invited collaborators can fork
Use Case	Sharing knowledge, Community contribution	Confidential development, Proprietary code

USE CASES – PRIVATE

- Company Projects**
Work on private and internal tools.
- Client Work**
Keep client code secure.
- Personal Ideas**
Work on secret or future projects.



KEY TAKEAWAY

Use **Public** repositories to share and learn together.

Use **Private** repositories to build and protect your ideas.



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **PUBLIC** — useful when others should view, learn from or contribute to the project.
- Examples: open-source learning projects, student portfolios, tutorials and examples.
- **PRIVATE** — useful when only you and selected collaborators should see the project.
- Examples: company work, client work, confidential ideas and internal projects.
- Rule: choose visibility based on project purpose and sensitivity.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



GitHub Use Cases

GitHub is more than just a code hosting platform.
It helps people learn, build, collaborate and grow in many ways!

1. EDUCATION



- Teachers can share teaching materials, code examples and assignments.
- Students can submit projects and collaborate with peers.
- Great for learning version control and real-world coding.

2. RESEARCH



- Researchers can share datasets, code, and findings openly.
- Supports collaboration across institutions and countries.
- Ensures transparency and reproducibility of research.

3. BUSINESS



- Teams can work together on projects efficiently.
- Helps in managing code, tracking changes and releasing updates.
- Improves productivity and project management.

4. PERSONAL PORTFOLIO



- Developers can showcase their projects and skills.
- Acts as an online portfolio for job opportunities.
- Shows your coding activity and contributions.

5. DOCUMENTATION



- GitHub allows creating beautiful documentation using Markdown.
- Projects can maintain user guides, FAQs, and manuals.
- Easy to update and collaborate.

BONUS: WHY USE GITHUB?



Cloud-based – Access from anywhere



Collaboration – Work with others easily



Safe & Secure – Version history is always saved



Open Source – Contribute to global projects



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **EDUCATION** — share teaching materials; students submit projects and collaborate.
- **RESEARCH** — share datasets, code and findings and support reproducibility.
- **BUSINESS** — manage code, track changes and coordinate team project work.
- The same platform supports learning, experimentation and professional teamwork.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

- **PERSONAL PORTFOLIO** — showcase projects, skills and coding activity.
- **DOCUMENTATION** — maintain README files, guides, FAQs and manuals.
- A well-organized repository communicates what a project does.
- GitHub can be both a project workspace and a professional showcase.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



ACTIVITY CREATE GITHUB ACCOUNT



GitHub is a platform where you can store your code, collaborate with others and build amazing projects!

1

Open a web browser.



2

Go to GitHub website.



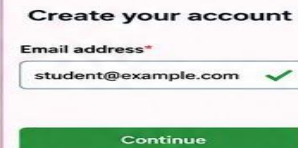
3

Click on Sign up.



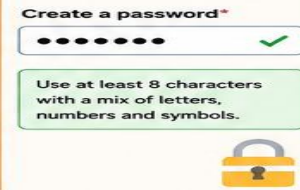
4

Enter your email address.



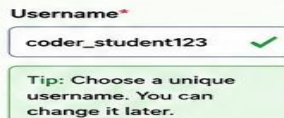
5

Create a strong password.



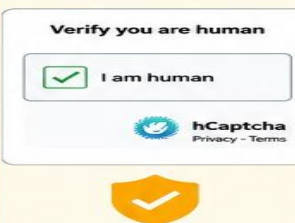
6

Choose a username.



7

Complete the verification.



8

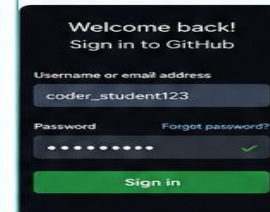
Verify your email address.



Check your inbox and click the verification link sent by GitHub.

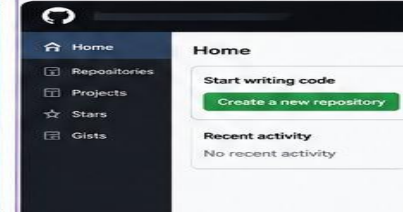
9

Log in to GitHub.



10

Explore your GitHub Dashboard.



Remember:

- ✓ Keep your username and password safe.
- ✓ Use a valid email address.
- ✓ You can use GitHub for your projects, assignments and collaborations.



Your Goal:

- ✓ Create your GitHub account
- ✓ Verify your email
- ✓ Explore the dashboard
- ✓ Start your coding journey!



Congratulations! You are now a part of the global developer community!



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



ACTIVITY EXPLORE GITHUB DASHBOARD

Let's explore the GitHub Dashboard – your home on GitHub!



1. HOW TO OPEN DASHBOARD

- 1 Open your browser and go to github.com
- 2 Sign in to your GitHub account
- 3 You will see your GitHub Dashboard (Home page)

DID YOU KNOW?
The Dashboard is your central place to access all your work, updates, and collaborations.

2. GITHUB DASHBOARD OVERVIEW

The screenshot shows the GitHub Dashboard interface with several key sections highlighted by callouts:

- Top Repositories:** Quick access to the repositories you work on most.
- News Feed (For you):** See updates from people and repositories you follow.
- Search Bar:** Search for repositories, issues, users and more.
- Profile Menu:** Access your profile, settings, and sign out.
- Latest changes:** Stay updated with new features and announcements.
- Explore Repositories:** Discover popular repositories on GitHub.
- Recent Activity:** Your recent actions will appear here.
- Discover & Explore:** Find interesting projects and people to follow.

3. WHAT YOU CAN DO FROM DASHBOARD

- Create a New Repository:** Start a new project.
- Collaborate with Others:** Invite, share and work together.
- Watch & Star Repositories:** Follow projects you like.
- Get Notifications:** Stay updated on activities.
- View Your Activity:** Check your contributions.
- Manage Your Profile:** Update profile, settings and more.

- TIPS**
- ✓ Explore each section of the dashboard.
 - ✓ Click on any repository to see its contents.
 - ✓ Use the search bar to find repositories, users or topics.
 - ✓ Make it a habit to check your dashboard regularly!

REMEMBER!

The GitHub Dashboard is the control center of your GitHub journey. Explore, learn and build amazing projects!



UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

TASK 1 — Open GitHub and sign in.

TASK 2 — Identify the dashboard, search bar, repositories and profile menu.

TASK 3 — Find one interesting repository.

TASK 4 — Identify whether it is Public or Private.

TASK 5 — Tell your partner one thing you learned from that repository.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

1. What problem does version control solve?

2. Is Git a tool or an online platform?

3. What is a repository?

4. What does a commit represent?

5. What is the main difference between Git and GitHub?

6. When would you choose a private repository?

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

VERSION CONTROL → keeps a history of changes.

GIT → version-control tool.

GITHUB → online platform for storing, sharing and collaborating.

REPOSITORY → project space containing files and history.

COMMIT → saved snapshot of changes.

BRANCH → separate line of work.

PUBLIC / PRIVATE → visibility choice based on purpose.

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

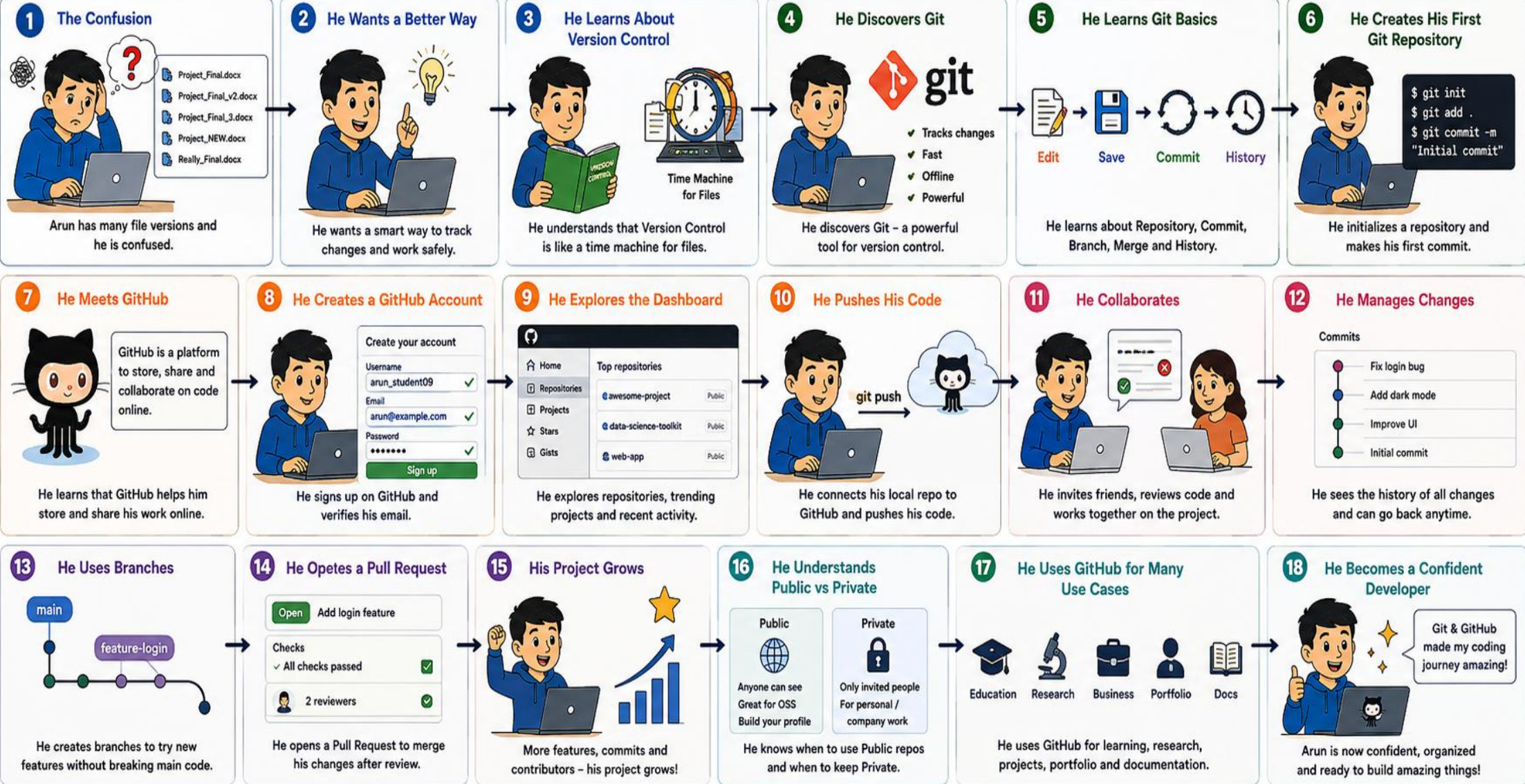
7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA



THE END (FOR NOW) – Arun's journey with Git & GitHub is just the beginning. Keep learning. Keep building. Keep growing! 🚀

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Activity 1: Create Account

8. Activity 2: Explore Dashboard

9. Public vs Private

10. Use Cases

← BACK TO AGENDA



WHAT IS GITHUB?



GitHub is an online platform where people can **store, manage, share, and work together** on software projects.



THINK OF GITHUB LIKE...

GitHub is like **Google Drive** for coding projects, but with special tools for programmers.



Store your code

+



Keep history of changes

+



Work together with others

+



Build amazing projects

KEY FEATURES OF GITHUB



Cloud-Based

Access your projects from anywhere, anytime.



Collaboration

Work with others on the same project in real-time.



Version Control

Track every change. Go back to any previous version.



Safe & Secure

Your code is safe in the cloud with backup.



Open Source

Share your projects with the world and learn from others.

IMPORTANT TERMS TO KNOW



Repository

A project folder where your code and files are stored.



Code

The files and programs you write.



Version History

A record of all the changes made to your project.



Collaboration

Working with others by sharing and making changes together.



Branch

A separate line of development to work on new features.



Pull Request

A request to review and merge your changes into the main project.



README

A file that explains what your project is about.

WHY USE GITHUB?

- ✓ Helps in organizing your projects.
- ✓ Makes teamwork easy.
- ✓ Tracks every change safely.
- ✓ Builds your portfolio as a developer.
- ✓ Used by millions of developers worldwide!



ONE-LINE TO REMEMBER

GitHub = A cloud-based platform for **storing, managing, sharing, and collaborating** on coding projects.

EXAMPLE

If 5 students are building an AI project:

- They put the project on GitHub.
- Each student works on different parts.
- They track changes and help each other.
- Finally, they combine everything into one complete project!



REMEMBER:

Write Code. Save on GitHub. Share. Collaborate. Succeed!

UNIT 1 TOPICS

1. Why Version Control

2. Introduction to Git

3. What is GitHub?

4. Git vs GitHub

5. GitHub Features

6. GitHub Interface

7. Public vs Private

8. Use Cases

9. Activity 1: Account

10. Activity 2: Dashboard

← AGENDA

READY FOR UNIT 2?

You now know the WHY, WHAT, and WHERE of Git & GitHub.

Next: hands-on Git and GitHub practice 🚀